

★ PRIMER · UNIT 8

Before the Lecture: One Neuron at a Time

Unit 8 · Neural Networks

Watch this first if weighted sums and slopes are hazy

Twenty minutes of groundwork — what one neuron computes, why a network needs a switch, and what walking downhill has to do with learning.

How to Use This Primer

Two audiences, one deck — read the column that describes you

If this is new to you

- Watch before the lecture.
- Weighted sums and slopes are here.
- Do slide 8's arithmetic.

If you have met it before

- Skim it — this is revision.
- Slide 7 is what finance changes.
- Slide 16 says if you can skip.

Nothing here is examined on its own — it is the floor the lecture stands on

Where Unit 8 Sits

The course builds one pipeline; here is this unit's stone

Unit 7

done

One tree, then a forest of them.

Unit 8

you are here

Neurons, layers and
backpropagation.

Unit 9

next

Finding groups with no answer
key.

Every neuron is unit 4's straight line with a switch bolted on the end.

What One Neuron Computes

A neuron multiplies each input by a weight, adds them up, adds a bias, and passes it through a switch.

The familiar part

- Weighted sum plus bias.
- This is unit 4's line.
- Nothing new so far.

The new part

- The sum meets a function.
- Negative becomes zero.
- That is the switch.

Without that switch a stack of layers collapses into one line, however deep it is.

The Words the Lecture Assumes

Five terms, and the whole of deep learning opens up

Word	What it means	Here
Weight	how much an input counts	learned, not chosen
Bias	a constant added at the end	shifts the switch point
Activation	the switch after the sum	ReLU: keep positives, zero the rest
Layer	a row of neurons	each builds on the last
Epoch	one pass over all the data	training runs many

ReLU is the whole of the switch: if the number is negative, make it zero.

What Each Layer Learns

Depth is not decoration — each layer builds on the last one's ideas

Layer	On an image	On market data
First	edges and corners	raw returns and volumes
Middle	shapes and textures	short-term patterns
Last	whole objects	a regime, or a verdict

That is what 'deep' means. Not bigger — layered, with each layer composing the one below.

Why Finance Is Hard for Deep Learning

Deep networks are hungry. They want millions of examples from a world whose rules do not change.

What they need

- Vast, clean datasets.
- Stable underlying rules.
- Images and text supply both.

What finance gives

- Sixty years of daily bars.
- Rules that shift each decade.
- Signal near the noise floor.

This is why unit 7's boosted trees still beat deep nets on most tabular financial problems.

Fire a Neuron by Hand

Three inputs, three weights, one bias — compute the output

Input	Value x	Weight w	$w \times x$
Income	0.8	1.5	1.20
Debt ratio	0.6	-2.0	-1.20
Years employed	0.4	0.5	0.20
sum	—	—	0.20
plus bias -0.5	—	—	-0.30

The sum is -0.30. Through ReLU that becomes 0. This neuron does not fire.

Why the Switch Has to Be There

Take it away and the depth buys you nothing at all

Without a switch

- Each layer is a line.
- A line of a line is a line.
- Ten layers equal one.

With ReLU

- Each layer can bend.
- Bends compose into curves.
- Now depth means something.

1969's famous complaint was exactly this — and the switch is the answer

What Learning Actually Is

No cleverness — just walking downhill, one small step at a time

The landscape

- Error as a function of weights.
- Hills are bad settings.
- Valleys are good ones.

The walk

- Find the downhill direction.
- Take one small step.
- Repeat a few million times.

The slope of the error is the gradient. Backpropagation is how you compute it cheaply.

One Training Step

Three moves, repeated until the error stops falling

1 · Forward

make a guess

Push the inputs through the layers and read the prediction.

2 · Compare

measure the miss

How far is the guess from the truth? That is the loss.

3 · Backward

nudge the weights

Work out which weight to move, and which way. Step a little.

Learning rate sets the step size. Too big and you bounce; too small and you never arrive.

Three Network Shapes

The lecture meets all three; each suits a different kind of row

Shape	Good at	In finance
Feed-forward	plain tables of features	credit scoring
Convolutional	grids — images, surfaces	volatility surfaces
LSTM	sequences with memory	order flow, conversations

The autoencoder is a fourth shape — it learns to rebuild its input, and flags what it cannot.

Notation You Will Meet

Four symbols carry the whole unit

Symbol	Read it as	It stands for
w	w	a weight — how much an input counts
b	b	the bias added before the switch
ReLU	rel-you	max of zero and the number
η	eta	the learning rate — the step size

Eta is the only dial in this unit you will set by hand. Everything else is learned.

The Python You Will See

You do not need to write it yet — you need to recognise it

Line	What it does
<code>nn.Linear(3, 1)</code>	one layer — three inputs, one output
<code>nn.ReLU()</code>	the switch
<code>loss = criterion(pred, y)</code>	how far the guess was
<code>loss.backward()</code>	work out every weight's slope
<code>optimizer.step()</code>	take one step downhill

Those last three lines are the entire training loop, in every PyTorch program ever written.

Three Things Beginners Get Wrong

Each one costs a week if you carry it into unit 9

1

"Deeper is always better"

Depth needs data. On sixty years of daily bars, a boosted tree usually wins.

2

"The activation is a technical detail"

Remove it and every layer collapses into one line. It is the reason depth exists.

3

"Neural networks are black boxes, so nothing can be said"

Unit 14 opens the box with SHAP. Opacity is a problem to solve, not an excuse.

If one of these surprised you, that is exactly what the primer was for

Self-Check Before the Lecture

Answer these three out loud — if you can, you are ready

1 Weighted sum 0.20, bias -0.5 . What does ReLU return?

0

2 Remove every activation. How much can ten layers do?

one line

3 What does the learning rate control?

step size

Cover the right column, answer, then check. All three come from this deck.

What the Lecture Does With This

You bring the vocabulary; the lecture brings the argument

1

It builds one neuron, then stacks them

From how you make a decision to a formula, with the neuron fired by hand.

2

It explains how learning happens

Gradient descent as walking downhill, and backpropagation as the cheap way to find the slope.

3

It ends in code

The problem set fires a neuron, solves XOR, and trains an autoencoder to flag anomalies.

Next: Unit 8 lecture — Neural Networks and Deep Learning